Reviewed by Nico Schmock (DA Mentor)
Masterschool-Student: Michael Klöß
Date: 16.09.2025
Project Evaluated: Horizon Project

Review: ✈️Horizon Project by Michael Klöß

File	Description	Status
GitHub Repository	Submit Date: 2025-09-14	Approved
Executive Project Summary	Submit Date: 2025-09-14	Approved
Video Presentation	Submit Date: 2025-09-14	Approved

✨ Overall Take

Michaels Horizon project submission is tidy, well-reasoned, and clearly put together. It frames a concrete business goal—assign each customer a likely favorite perk for a personalized rewards invite—and builds an end-to-end path to get there: SQL for extraction and enrichment, Python/pandas for aggregation and scoring, Matplotlib for quick sanity checks, and Jupyter notebooks to stitch it all together. The repo is unusually well documented—there's a full docs set, processed datasets, and polished reports—which makes the work easy to follow and reuse. The stack is **Postgres + SQL + Python (pandas/Matplotlib) + Jupyter Notebooks.**

🧭 Strategic Framing & Cohort

The analysis anchors on a **high-engagement cohort**: users with **more than 7 sessions between Jan 5, 2023 and Jul 28, 2023**. The thinking is solid—enough interactions per user to say something meaningful about behavior, aligned with a recent product/marketing window. The trade-offs (threshold arbitrariness, temporal bias) are noted, with sensitivity checks proposed. Net: a coherent, defensible starting point that can be validated and tuned as results come in.

🧰 Data Prep that Scales (SQL → Python)

The SQL pipeline is modular and deliberate. It filters the time window, builds the **>7-sessions** user list, fixes **reversed hotel check-in/out timestamps** (to correct negative "nights"), derives **stay duration**, and assembles a **session-level enriched table** by joining users, flights, and hotels. This layer adds practical fields for downstream logic: **binary flags**, **age from birthdate**, **Haversine flight distance**, **parsed hotel name/location**, and **trip-level cancellation** via window functions. Hygiene is good—consistent LEFT JOINs to preserve grain, `COALESCE/NULLIF` use, and explicit temporal checks.

Reviewed by Nico Schmock (DA Mentor)
Masterschool-Student: Michael Klöß
Date: 16.09.2025
Project Evaluated: Horizon Project

From there, the pipeline moves into pandas. The choice to run the analysis in **notebooks** is explicit and well organized (exploration → cleansing → preprocessing → modeling). In pandas, the code aggregates to **user level**, engineers summary features, handles missingness/outliers, and prepares the final **perk-assignment** table. The SQL → Python handoff is clean: minimal redundancy, clear intent, and each tool used where it's strongest.

🧪 Feature Engineering & Segmentation Logic

Feature engineering is a standout. The project ships **35+ metrics** spanning spend, travel dynamics, and promo sensitivity—examples include conversion ratio, average nights, mean checked bags, promo-booking ratio, flight:hotel mix, booking lead times, and even **savings per kilometer** (distance-normalized) to reduce bias from long-haul trips. These are the sorts of features marketers can actually act on, not just model fodder.

On top of that sits a **hierarchical, rule-based classifier** that assigns each user to **one of seven segments** (Luxury, Business, Family, Young, Spontaneous, Budget, Lost Opportunities). Each class has transparent thresholds (e.g., **Luxury** via high per-trip value; **Business** via short stays + frequency + flight/hotel mix; **Budget** via lower per-trip value + higher promo ratio). The hierarchy resolves overlaps deterministically, which is exactly what you want when the product needs **one** perk per user. It's easy to explain, easy to QA, and easy to iterate.

📓 Notebooks & Documentation Quality

The documentation is excellent. There are standalone docs for **business context**, **data prep**, **cohort definition**, **feature engineering** (with business rationale), **segmentation spec**, and even a **bias-injection log** that spells out where domain assumptions enter and how they'll be validated or toned down. The project structure file sets expectations on what's complete vs. pending (e.g., evaluation awaits A/B infrastructure). This level of transparency makes the work genuinely production-friendly.

🔎 Findings that Matter (and a Flag to Validate)

Two notes worth highlighting:

- **Discounts & cancellations**: the data story calls out that ~90k cancellations are linked with both flight and hotel discounts. If that linkage is truly universal, it's a red flag that likely points to a data quirk (join logic, trip grain, or how discounts are recorded). Surfacing it is the right move—now it needs a tight SQL repro at **trip level** to confirm or fix.

Reviewed by Nico Schmock (DA Mentor)
Masterschool-Student: Michael Klöß
Date: 16.09.2025
Project Evaluated: Horizon Project

- **Scale & cleanliness**: anomalies like negative nights are addressed (timestamp reorder), and browse-heavy sessions are acknowledged as noise to control for. This sets realistic expectations on reliability per segment.

🧭 Recommendations (Product + Analytics)

Short term:
Run an **A/B test** that compares generic messaging vs. **segment-specific perks**. Measure conversion, incremental revenue, and short-term retention. Keep the first test simple (no micro-segments) to get clean reads. In parallel, **re-validate the discount–cancellation linkage** with a minimal trip-grain SQL query; document the cause and adjust features if needed.

Medium term:
Do a **cohort sensitivity sweep** (e.g., 5/10/15-session thresholds; optional quality filter like ≥30s session duration) and report changes in segment sizes/KPIs. Continue updating the **bias log**—pause or de-emphasize any hotel-brand or airline heuristics until validated by data.

Operationally:
Ship with a **Go/No-Go framework** (already drafted), plus monitoring for **segment drift**, **distribution health**, and **perk delivery feasibility** so marketing isn't promising perks the system can't consistently fulfill.

🏁 Final Verdict

Informed, integrated, and pragmatic. Michaels project picks a clear objective, builds a clean SQL→Python pipeline, engineers features that matter for marketing, and wraps everything in documentation a team can ship. The **rule hierarchy** for seven segments is transparent and testable, and the repo clearly states what's done vs. what needs validation. The stack—**SQL + Python + Matplotlib + Jupyter Notebooks**—is used where each tool adds the most value.

If you want a portfolio piece that looks and feels ready for real-world testing, this is it. It's thoughtful, production-minded analytics that's ready to iterate as results come in.